

YallaJo Sequenced Development Plan (Phase 1 & 2)



IMPORTANT: Endpoints are ordered logically by data dependency. Do not start a Wave until the dependencies in the previous Wave are complete. Within each Wave, tasks are distributed by difficulty (Easy ●, Medium ●, Hard ●).

Wave 1: The Absolute Foundations

No dependencies. Can be built immediately. Focuses on Auth, Categories, and Base Profiles.

● Tier 1 (Easy)

- ☐ GET /profile — Get current user's profile
- ☐ GET /categories — List categories (tree structure, with translations)
 - **3-level hierarchy:** Root → SubCategory → Sub-SubCategory (max depth enforced)
 - Return as tree structure: parent with nested children
 - Include translations for request's Accept-Language
 - Filter: isActive, parentId (to get children of a specific category)
 - Slugs must be unique per level, auto-generated from name if not provided

☐ GET /categories/{id} — Get single category with subcategories

☐ DELETE /categories/{id} — Soft-delete category

☐ GET /languages — List all active languages

- Return only active languages (IsActive = true)
- Include: code (ISO 639-1), name, nativeName, isRtl
- Used by frontend for language picker and translation UI
- Cache: 1 hour (languages rarely change)

☐ GET /roles — List all roles

☐ GET /roles/{id}/claims — Get claims for a role

☐ DELETE /roles/{id} — Delete role

☐ GET /specializations — List all specializations

☐ GET /tags — List all tags (filterable, paginated)

☐ GET /users — List all users (paginated, filterable)

- Admin only — requires Admin role claim
- Support filters: email (partial match), role, status (active/inactive/suspended), registration date range
- Pagination: page + pageSize (max 50), sortable by name, email, createdAt
- Return: userId, email, roles, status, lastLoginAt, createdAt (no passwords or sensitive data)

☐ GET /users/{id} — Get user details by ID

● Tier 2 (Medium)

☐ POST /change-password — Change password (requires current password)

- Require current password verification via `ISecurityService.VerifyCredentialsAsync()`
- Validate new password strength, must differ from current password
- Update password, optionally revoke other sessions (user choice)

☐ POST /external/apple — OAuth login/register via Apple

☐ POST /external/facebook — OAuth login/register via Facebook

☐ POST /external/google — OAuth login/register via Google

☐ POST /forgot-password — Initiate password reset → sends OTP to email

- Lookup user by email via `ISecurityService`
- If user exists: generate 6-digit OTP, store with 10-min TTL, send via email
- If user doesn't exist: return 200 anyway (prevent email enumeration)
- Rate limit: max 5 requests per email per hour

☐ POST /login — Email + password login → returns tokens

- Normalize email, call `ISecurityService.VerifyCredentialsAsync()`
- Return 401 if credentials invalid or email not verified
- Create Device record (user agent, device name from headers)
- Create Session (30-day expiry, IP address from `IRequestContext`)

- Generate refresh token, hash it, store in `auth.RefreshTokens` (30-day expiry)
- Generate JWT access token with claims: `sub` (`userId`), `email`, `roles`, custom claims
- JWT expiry: 15 minutes (short-lived)
- Track: last login timestamp, login count

☐ POST `/refresh` — Refresh access token

- Hash incoming refresh token, look up in `auth.RefreshTokens`
- Validate: not revoked, not expired, session still active
- Rotate: revoke old refresh token, generate new one (refresh token rotation for security)
- Generate new JWT access token with fresh claims from `ISecurityService.GetUserDataByIdAsync()`
- If refresh token reuse detected (already revoked token presented): revoke ALL user sessions (security breach)

☐ POST `/register` — Register new user (email + password) → sends OTP

- Validate: email format, password minimum 8 chars with uppercase + lowercase + digit + special character
- Normalize email: `Trim().ToLowerInvariant()`
- Check uniqueness: call `ISecurityService.GetUserIdByEmailAsync()` — return 409 Conflict if exists
- Create user in Security module via `ISecurityService` (hashed password, default role `User`)

- Generate 6-digit OTP, store in `auth.0tps` with 10-minute TTL
- Send OTP via email using `IService`
- Publish `UserCreatedIntegrationEvent` → Accounts module creates Profile
- Rate limit: max 3 registration attempts per IP per hour

☐ `POST /resend-otp` — Resend OTP to email (rate limited: 5/hour)

- Rate limit: max 5 per email per hour (enforced via rate limiting middleware)
- Invalidate previous OTP for this email
- Generate new 6-digit OTP with 10-min TTL, send via email

☐ `POST /reset-password` — Reset password with OTP verification

- Validate OTP (same rules as `verify-email`)
- Validate new password (same strength rules as `register`)
- Update password via `ISecurityService`, revoke ALL sessions and refresh tokens
- Send confirmation email

☐ `POST /verify-email` — Verify email with OTP code → returns tokens

- Lookup OTP by email + code in `auth.0tps`
- Check: not expired (10-min TTL), not already used, max 5 attempts
- On success: call `ISecurityService.MarkEmailVerifiedAsync()`, generate JWT + refresh token
- On failure: increment attempt count, return 404 if OTP not found, 429 if max attempts exceeded

- Delete OTP after successful verification

☐ POST /categories — Create category (with parent for hierarchy)

- Validate: name (2-200 chars), slug (unique, URL-safe), parentCategoryId exists if provided
- Enforce max 3 levels: if parent is a level-2 category, reject (would create level-4)
- Auto-generate slug from name using transliteration (Arabic → Latin)
- Set SortOrder: next available in parent's children
- Auto-trigger translation: publish ContentCreatedIntegrationEvent for auto-translation

☐ PUT /categories/reorder — Reorder categories (batch sort order update)

☐ PUT /categories/{id} — Update category

☐ POST /languages — Add a new language

☐ PUT /languages/{id} — Update language (name, RTL, active status)

☐ POST /roles — Create a new role

- Role name must be unique (case-insensitive)
- Reserved names: Admin, User, Provider, SuperAdmin — cannot create

☐ POST /roles/{id}/claims — Add claim to role

- Claims structure: Type + Value pairs
- Standard claim types: permission, feature, tier
- Used for feature gating (subscription tier enforcement)

☐ PUT /roles/{id} — Update role

☐ POST /specializations — Create specialization

☐ PUT /specializations/{id} — Update specialization

☐ POST /tags — Create tag

☐ PUT /tags/{id} — Update tag

☐ POST /users/{id}/roles — Assign role to user

- Validate role exists before assignment
- System roles (Admin , User , Provider) cannot be deleted
- Provider role assignment triggers provider profile setup flow
- Removing Provider role: check no active bookings or pending payouts before removal

☐ DELETE /users/{id}/roles/{roleName} — Remove role from user

☐ PUT /users/{id}/status — Activate/deactivate user

- Toggle user active/inactive. Deactivated users cannot login
- If deactivating: revoke all sessions and refresh tokens for this user
- Cannot deactivate yourself (prevent admin logout)
- If reactivating: user must re-verify email if previously suspended for policy violations
- Publish UserStatusChangedIntegrationEvent → other modules react (cancel pending bookings, etc.)

● Tier 3 (Hard)

- ☐ POST /logout — Revoke refresh token + session
 - ☐ POST /logout-all — Revoke all sessions for user
 - ☐ GET /sessions — List active sessions for current user
 - Return all active (non-expired, non-revoked) sessions for `ICurrentUser.UserId`
 - Include: device info, IP address, created at, last activity, is current session
 - ☐ DELETE /sessions/{id} — Revoke a specific session
 - Validate session belongs to `ICurrentUser.UserId`
 - Revoke session: mark as revoked, revoke associated refresh token
 - Cannot revoke own current session (use /logout instead)
-

Wave 2: First-Level Content & Onboarding

Depends on Wave 1. Introduces Places, Attachments, and Provider registration workflows.

● Tier 1 (Easy)

- ☐ GET /admin/providers — List all provider applications (paginated)
- ☐ GET /provider/status — Get provider registration status (state machine)

- GET /attachments/{entityType}/{entityId} — Get all attachments for an entity
- GET / — List places (paginated, filterable by category, location, rating)
 - Public endpoint, supports:
 - Filter by: categoryId, coordinates (lat/lng/radius), rating range, hasActiveTours
 - Sort by: name, rating, popularity, distance (if coordinates provided)
 - Pagination: page + pageSize (max 50)
 - Return with: primary image, average rating, tour count, translations (per Accept-Language)
 - Apply soft-delete filter (IsDeleted = false)
- GET /{id} — Get place details (with translations, images, businesses)
- DELETE /{id} — Soft-delete place
- GET /{slug} — Get place by slug (SEO-friendly URL)

● Tier 2 (Medium)

- POST /admin/providers/{id}/approve — Approve provider application
 - Transition: Pending → Approved
 - Assign Provider role to user via ISecurityService
 - Publish ProviderApprovedIntegrationEvent → enable provider features, send welcome email

- Provider can now create tours, manage availability, receive bookings

☐ POST /admin/providers/{id}/reject — Reject provider application (with reason)

- Transition: Pending → Rejected
- Requires: rejectionReason (string, mandatory)
- Notify provider via email + in-app notification with reason
- Provider can fix issues and re-apply (#10)

☐ POST /admin/providers/{id}/request-docs — Request additional documents

- Transition: Pending → MoreDocsNeeded
- Requires: list of missing/insufficient document types with notes
- Notify provider with specific document requests

☐ POST /admin/providers/{id}/suspend — Suspend active provider

- Transition: Approved → Suspended
- Requires: suspensionReason
- Effects: provider cannot receive new bookings, existing confirmed bookings proceed
- Publish ProviderSuspendedIntegrationEvent → hide provider's tours from search, notify booked users

☐ PUT /profile — Update profile (name, bio, avatar, preferences)

- Updatable fields: firstName, lastName, bio (max 500 chars), dateOfBirth, gender, phoneNumber, preferredLanguage, preferredCurrency (JOD/USD/EUR)
- Validate phone format (E.164), language code exists in Languages table
- Cannot change email (handled by Auth module)
- UpdatedAt timestamp auto-set

☐ POST /profile/avatar — Upload profile avatar

- Accept: JPEG, PNG, WebP — max 5MB
- Resize to standard sizes: 64x64 (thumbnail), 200x200 (small), 400x400 (medium)
- Store via attachment service, update profile AvatarUrl
- Delete previous avatar files on replacement

☐ POST /provider/apply — Submit provider application for review

- Validates all required documents are uploaded for the provider type
- All documents must be non-expired
- Transitions state to Pending (if was MoreDocsNeeded or Rejected)
- Sends notification to admin queue

☐ POST /provider/documents — Upload provider verification documents

- **Required Documents** (by provider type):
- All: Business license, tax registration
- TourOperator: Tourism authority license, insurance certificate

- IndependentGuide: Personal ID, first aid certification
- HotelResort: Health & safety certificate, fire safety
- ActivityCenter: Activity-specific certifications, liability insurance
- File constraints: PDF, JPEG, PNG — max 10MB per document
- Each document has: type, file, expiresAt (mandatory for licenses/certifications)
- Documents stored encrypted (IV + HMAC in Attachment record)
- **Document Expiry:** DocumentExpiryCheckService runs daily, flags expiring docs 30 days before, suspends provider if critical doc expires

☐ PUT /provider/documents/{id} — Re-upload expired/rejected document

☐ POST /provider/register — Register as service provider
(TourOperator/IndependentGuide/HotelResort/ActivityCenter)

- **Provider Types:** TourOperator, IndependentGuide, HotelResort, ActivityCenter
- Required fields: businessName, providerType, contactEmail, contactPhone, address, description
- Additional by type:
 - TourOperator: licenseNumber, registrationNumber
 - IndependentGuide: personalIdNumber, languages spoken
 - HotelResort: starRating, facilityType
 - ActivityCenter: activityTypes, certifications

- Initial state: Pending (state machine begins)
- Publish `ProviderRegisteredIntegrationEvent`

☐ `POST /attachments` — Upload attachment (image, video, document)

- File upload with validation:
- Images: JPEG, PNG, WebP, AVIF — max 10MB, auto-resize to standard sizes
- Videos: MP4, MOV — max 100MB, extract thumbnail
- Documents: PDF — max 20MB
- Store encrypted: generate IV + HMAC for each file
- Track: `originalFileName`, `mimeType`, `fileSize`, `width`, `height`, `durationSeconds`
- `EntityType` + `EntityId`: polymorphic association (Place, Tour, Business, Review, Blog, TourGuide)

☐ `DELETE /attachments/{id}` — Delete attachment

☐ `POST /attachments/{entityType}/{entityId}/images` — Upload and assign images to entity

- Upload multiple images at once (max 20 per entity)
- Auto-create `EntityImage` records for each size (Thumbnail, Small, Medium, Large, Original)
- First image is auto-set as primary (`IsPrimary = true`)
- Validate: caller owns the entity (provider owns tour, admin for places)

☐ `PUT /attachments/{entityType}/{entityId}/images/reorder` — Reorder images, set primary

☐ POST / — Create a new place

- Admin only — places are curated content
- Required: name, slug (unique), location (lat, lng, address), description, categoryId
- Optional: placeType, elevation, timezone, website, phone, email
- Auto-trigger: translation for all active languages, sitemap regeneration
- Publish PlaceCreatedIntegrationEvent

☐ PUT /{id} — Update place

Wave 3: Core Catalog (Places & Tours)

Depends on Wave 2. Introduces the actual Tours, Businesses, and Tour Guides.

Tier 1 (Easy)

☐ GET /{id}/businesses — List businesses at a place

☐ GET /businesses/{id}/hours — Get business hours

☐ GET /guides — List tour guides (filterable by language, specialization, rating)

☐ GET /guides/{id} — Get tour guide profile

☐ GET / — List tours (paginated, filterable by category, place, price, date, rating)

- Public — only returns tours with Status = Approved and IsActive = true
- Filters: categoryId, placeId, priceRange (min/max), dateRange, durationRange, languageCode, childrenFriendly, accessibilityLevel, guidId
- Sort: price (asc/desc), rating, popularity, newest, distance (if coordinates)
- Pagination: page + pageSize (max 50)
- Include: primary image, starting price, average rating, review count, next available date
- ☐ GET /{id} — Get tour details (with translations, schedules, pricing, images)
- ☐ DELETE /{id} — Soft-delete tour
- ☐ GET /{slug} — Get tour by slug (SEO-friendly)

● Tier 2 (Medium)

- ☐ POST /businesses — Create business listing
 - Provider creates a business listing associated with a place
 - Required: name, placeId, businessType, contactInfo, description
 - Must verify: place exists and is active
 - Business requires admin approval before appearing publicly (optional workflow)
 - Auto-create default business hours (Mon-Fri 9-5) that provider customizes
- ☐ DELETE /businesses/{id} — Soft-delete business

- ☐ PUT /businesses/{id} — Update business listing
- ☐ PUT /businesses/{id}/hours — Set/update business hours (batch)
 - Batch update: send all 7 days at once (replace strategy, not merge)
 - Each day: dayOfWeek, openTime (TimeOnly), closeTime (TimeOnly), isClosed (bool)
 - Validation: openTime < closeTime (unless 24h: both 00:00), no overlapping shifts per day
 - Support: split shifts (two entries per day — e.g., 9:00-13:00, 17:00-22:00)
- ☐ POST /guides — Register as tour guide (provider creates guide profile)
- ☐ DELETE /guides/{id}/languages/{langId} — Remove language from guide
- ☐ POST /guides/{id}/languages — Add language to guide
- ☐ POST /guides/{id}/specializations — Add specialization to guide
- ☐ PUT /guides/{id} — Update tour guide profile
- ☐ POST / — Create tour (draft status)
 - Provider only — must have Status = Approved provider account
 - Initial tour status: Draft
 - Required: title, description, placeId, categoryId, durationMinutes, basePrice, currency (JOD/USD/EUR), maxGroupSize, languageCode
 - Optional: minGroupSize, difficultyLevel, ageRestriction, meetingPoint (lat, lng, address), whatToExpect, whatToBring, cancellationPolicy

- Validate: placeId exists, categoryId exists, price > 0, duration > 0
- Auto-trigger translation upon publishing

☐ PUT /{id} — Update tour details

Wave 4: Deep Content & Enrichment

Depends on Wave 3. Adds schedules, pricing, blogs, SEO, and translation hooks to the core entities.

Tier 1 (Easy)

- ☐ GET /{id}/children-info — Get children-friendly details (age ranges, facilities)
- ☐ GET /{id}/pricing — Get pricing tiers for a tour
- ☐ GET /{id}/schedules — Get tour schedules/available dates
- ☐ GET /{id}/waypoints — Get tour route waypoints
- ☐ GET / — List published blog posts (paginated, filterable by tag, category, author)
- ☐ GET /{id} — Get blog post details (with translations, comments, related tours)
- ☐ GET /{slug} — Get blog by slug (SEO-friendly)
- ☐ GET /faq/{entityType}/{entityId} — Get FAQ items for entity (with translations)

- ☐ GET /metadata/{entityType}/{entityId} — Get SEO metadata for entity
- ☐ GET /redirects — List all redirects
- ☐ GET /translations/{entityType}/{entityId} — Get all translations for entity

● Tier 2 (Medium)

- ☐ POST /admin/{id}/approve — Approve tour (Pending → Approved)
- ☐ POST /admin/{id}/reject — Reject tour (with reason)
- ☐ PUT /{id}/children-info — Update children-friendly metadata
- ☐ POST /{id}/pricing — Create pricing tier (adult, child, group, etc.)
 - Pricing tier types: Adult , Child , Infant , Senior , Group , Private
 - Each tier: type, price, currency, minAge, maxAge, minQuantity, maxQuantity
 - At least one Adult tier required
 - Child tier: must have age range (e.g., 3-12), price can be 0 (free)
 - Group tier: special pricing when minQuantity reached
 - Sale price: separate field, calculated by DiscountLifecycleService when discounts apply
- ☐ PUT /{id}/pricing/{tierId} — Update pricing tier
- ☐ POST /{id}/schedules — Create schedule (date, time, capacity)
 - Create availability schedule: date, startTime, endTime, capacity
 - Recurrence support: once , daily , weekly , custom (specific days of week)

- For recurring: generate slots up to 90 days ahead
- Validation: no overlapping schedules for same tour, capacity > 0
- Blackout dates: can mark specific dates as unavailable

☐ DELETE /{id}/schedules/{scheduleId} — Delete schedule (if no bookings)

☐ PUT /{id}/schedules/{scheduleId} — Update schedule

☐ POST /{id}/submit — Submit tour for approval (Draft → Pending)

- Transition: Draft → Pending
- Pre-submit validation:
 - At least 1 image uploaded
 - At least 1 pricing tier defined
 - At least 1 schedule defined
 - Description > 100 characters
 - Meeting point coordinates set
- Fail with 422 and list of missing requirements if validation fails

☐ POST /{id}/waypoints — Add waypoint to tour route

☐ DELETE /{id}/waypoints/{waypointId} — Remove waypoint

☐ PUT /{id}/waypoints/reorder — Reorder waypoints

☐ POST / — Create blog post (draft status)

- Admin/editor only — content team creates blog posts

- Initial status: Draft
- Required: title, slug (unique), body (rich text/markdown), authorId, categoryId
- Optional: featuredImageId, tags[], excerpt (auto-generated from first 200 chars if empty)
- SEO: auto-generate meta description from excerpt, og:image from featured image

☐ PUT /{id} — Update blog post

☐ DELETE /{id} — Soft-delete blog post

☐ POST /{id}/publish — Publish draft blog post

- Transition: Draft → Published
- Sets publishedAt timestamp (for chronological listing)
- Auto-trigger: translation, sitemap regeneration, SEO metadata creation
- Publish BlogPublishedIntegrationEvent (for notification to subscribers)

☐ POST /{id}/tours — Associate tour with blog post

☐ DELETE /{id}/tours/{tourId} — Remove tour association

☐ POST /{id}/unpublish — Unpublish blog post (back to draft)

☐ POST /faq — Create FAQ item

☐ DELETE /faq/{id} — Delete FAQ item

☐ PUT /faq/{id} — Update FAQ item

☐ PUT /faq/reorder — Reorder FAQ items

☐ POST /metadata — Create/update SEO metadata (title, description, keywords, og:tags)

- One SEO record per entityType + entityId (upsert)
- Fields: metaTitle (max 60 chars), metaDescription (max 160 chars), keywords[], ogTitle, ogDescription, ogImageUrl, canonicalUrl, noIndex (bool), hreflang links
- Auto-populated defaults from entity data if not manually set
- Language variants: separate SEO metadata per language (hreflang)

☐ PUT /metadata/{id} — Update SEO metadata

☐ POST /redirects — Create URL redirect (301/302)

- Create 301 (permanent) or 302 (temporary) redirect
- Validate: oldPath is unique (no duplicate redirect sources), newPath exists or is valid URL
- Circular redirect detection: follow chain, reject if loop
- Redirect chain limit: max 3 hops

☐ DELETE /redirects/{id} — Delete redirect

☐ POST /translations/batch — Batch translate multiple texts

☐ POST /translations/translate — On-demand translation (admin tool)

☐ POST /translations/{id}/approve — Mark translation as human-reviewed

☐ PUT /translations/{id} — Update/override a translation

Wave 5: The Booking & Payment Engine

Depends on Wave 4. The critical financial tier. Creates availability slots, handles concurrency, and processes payments.

Tier 1 (Easy)

- ☐ GET /availability/{tourId} — Get available dates/slots for a tour
- ☐ GET /availability/{tourId}/{date} — Get slots for specific date (with remaining capacity)
- ☐ GET /{id} — Get booking details (with payment status, provider info)
- ☐ GET /admin/all — List all bookings (admin view, filterable)
- ☐ GET /my-bookings — List current user's bookings (paginated, filterable by status)
- ☐ GET /provider/history — Provider booking history
- ☐ GET /provider/pending — List pending bookings for provider
- ☐ GET /provider/upcoming — List upcoming confirmed bookings for provider
- ☐ GET /refund-policies/{tourId} — Get refund policy for a tour
- ☐ GET /commissions — List commission rules
- ☐ GET /invoices/my-invoices — List user's invoices
- ☐ GET /invoices/provider/my-invoices — List provider's invoices
- ☐ GET /invoices/{id} — Get invoice details with line items

- ☐ GET /invoices/{id}/download — Download invoice as PDF
- ☐ GET /payments/admin/all — List all payments (admin)
- ☐ GET /payments/my-payments — List user's payment history
- ☐ GET /payments/{id} — Get payment details

● Tier 2 (Medium)

- ☐ POST /availability/slots — Create availability slot
 - Provider creates time slots for their tours
 - Validation: no overlap with existing slots for same tour + date
 - Capacity: must be ≥ 1 , cannot exceed tour's maxGroupSize
 - Supports: single slot creation or recurring via bulk endpoint
- ☐ DELETE /availability/slots/{id} — Delete slot (if no bookings)
- ☐ PUT /availability/slots/{id} — Update slot (capacity, time)
- ☐ POST /{id}/complete — Mark booking as completed (after tour)
- ☐ POST /join-request — Request to join an existing group booking
 - Request to join existing confirmed group booking (if tour allows)
 - Validate: booking is Confirmed, tour allows join, capacity remaining
 - Join request state: Pending → Approved / Rejected
 - If approved: add participant, charge separately, update booking count
- ☐ POST /join-request/{id}/approve — Approve join request

☐ POST /join-request/{id}/reject — Reject join request

☐ POST /refund-policies — Create refund policy

- Per-tour refund policy (provider sets when creating tour)
- Structure: list of tiers with `hoursBeforeTour` threshold and `refundPercentage`
- Example: [{hours: 48, percent: 100}, {hours: 24, percent: 50}, {hours: 0, percent: 0}]
- Default policy if none set: full refund up to 24h, 0% after

☐ PUT /refund-policies/{id} — Update refund policy

☐ POST /commissions — Create commission rule (tiered by revenue)

☐ DELETE /commissions/{id} — Delete commission rule

☐ PUT /commissions/{id} — Update commission rule

● Tier 3 (Hard)

☐ POST /availability/slots/bulk — Bulk create recurring availability slots

- Generate recurring slots: specify pattern (daily/weekly/custom), start date, end date, time, capacity
- Max: generate up to 90 days of slots at once
- Skip existing dates (no duplicate slots)
- Return: count of created slots + any skipped dates

☐ POST /{id}/cancel — Cancel booking (trigger refund if applicable)

- User cancels own booking – refund calculated by refund policy:
- $\geq 48h$ before tour: full refund
- 24-48h before: 50% refund (configurable per tour)
- $< 24h$ before: no refund (configurable per tour)
- Provider cancels: always full refund to user
- Release slot lock, increment available capacity
- Publish `BookingCancelledIntegrationEvent` → trigger refund processing, notify parties

☐ `POST /{id}/confirm` – Provider confirms non-instant booking

- Only for non-instant bookings in `PendingConfirmation` status
- Provider must confirm within 24h (else `ProviderAutoAcceptService` auto-confirms)
- Transition: `PendingConfirmation` → `Confirmed`
- Notify user via `NotificationHub` + email

☐ `POST /{id}/reject` – Provider rejects non-instant booking (with reason)

- Transition: `Pending` → `Rejected`
- Requires: `rejectionReason` (string, mandatory)
- Notify provider via email + in-app notification with reason
- Provider can fix issues and re-apply (#10)

☐ `POST /tour` – Create tour booking (lock slot → `AwaitingPayment`)

- **Step 1 – Validate availability:**

- Check tour exists, is Approved, is Active
- Check schedule/slot exists for requested date, has remaining capacity
- Verify requested participant count $\leq$ remaining capacity
- Check: user has no duplicate booking for same tour + date

- **Step 2 – Lock slot** (optimistic concurrency):

- Create SlotLock record: bookingId, slotId, userId, expiresAt = Now + 10 min
- Decrement available capacity atomically (use RowVersion for concurrency)
- If capacity insufficient after lock: return 409 Conflict (race condition)

- **Step 3 – Calculate pricing:**

- Sum pricing tiers $\times$ quantities (adults, children, etc.)
- Apply active discounts (check Discounts table: valid date range, not exhausted, applicable to this tour)
- Apply discount stacking rules: max 1 percentage + 1 fixed, or 1 early bird, highest value wins if conflict
- Apply loyalty points if user redeems (deduct from balance)
- Currency: use tour's base currency (JOD/USD/EUR)
- Calculate commission: apply CommissionRules (tiered by provider's total revenue)

- **Step 4 – Create booking:**

- Status: `AwaitingPayment`

- Generate booking reference number (format: `YJ-{YYYYMMDD}-{random6}`)

☐ `POST /payments/initiate` — Initiate payment for booking (returns payment URL/token)

- Input: `bookingId`, `paymentMethod` (card, wallet, etc.)
- Verify booking exists, is in `AwaitingPayment` status, belongs to user, not expired
- Calculate: `totalAmount`, `currency`, `commission breakdown`
- **Escrow model:** payment goes to YallaJo escrow, NOT directly to provider
- Provider payout happens after tour completion (weekly batch)
- Call payment gateway (abstracted behind `IPaymentGateway`):
- Returns: `paymentUrl` (redirect) or `clientSecret` (Stripe Elements)
- Create `Payment` record: `amount`, `currency`, `gatewayRef`, `status= Pending`
- Return payment token/URL to frontend

☐ `POST /payments/webhook` — Payment gateway webhook (confirm/fail)

- Payment gateway callback (Stripe/PayMob webhook)
- Verify webhook signature (gateway-specific)
- On success:
- Update `Payment` status: `Pending` → `Completed`
- Transition booking: `AwaitingPayment` → `Confirmed` (instant) or `PendingConfirmation` (non-instant)

- Release slot lock, permanently reserve capacity
- Create invoice with line items
- Publish `PaymentCompletedIntegrationEvent` → notify user + provider, award loyalty points
- On failure:
- Update Payment status: `Pending` → `Failed`
- Release slot lock, restore capacity
- Notify user of payment failure

☐ `POST /payments/{id}/refund` — Initiate refund (full or partial)

- Initiate refund (full or partial based on refund policy)
- Call `IPaymentGateway.RefundAsync(gatewayRef, amount)`
- If gateway call fails: mark as `RefundPending`, `RefundRetryService` retries every 15 min (max 3 attempts)
- On success: create `Payment` record with negative amount (refund), update booking
- Commission handling: YallaJo may retain commission on refunds (configurable per policy)

☐ `GET /payouts/admin/pending` — List pending payouts

☐ `GET /payouts/provider` — List provider's payouts

☐ `GET /payouts/{id}` — Get payout details with items

☐ `POST /payouts/admin/trigger` — Manually trigger payout batch

- Admin manually triggers payout batch (normally weekly via `PayoutBatchingService`)
- Aggregation logic:
- Group completed bookings by provider + currency
- Calculate: $\text{totalBookingRevenue} - \text{commission} = \text{netProviderAmount}$
- Commission tiers (from `CommissionRules`):
- Revenue $\leq 10\text{K}$ JOD/month: 15% commission
- Revenue 10K-50K: 12% commission
- Revenue $> 50\text{K}$: 10% commission
- Deduct: any pending refunds, dispute holds
- Create `Payout` + `PayoutItem` records
- Status: Pending (requires admin approval for large payouts $> 5\text{K}$)

☐ POST `/payouts/{id}/approve` — Approve payout for processing

Wave 6: Post-Booking, Discovery & Background Jobs

Depends on Wave 5. Features that require completed bookings (Reviews), Search algorithms, Maps, and automated Background Services.

Tier 1 (Easy)

- ☐ GET /provider/dashboard — Provider dashboard summary (bookings, earnings, rating)
- ☐ GET /featured — Get featured/promoted tours
- ☐ GET /provider/my-tours — List current provider's tours
- ☐ GET /{id}/comments — List comments on blog post (paginated)
- ☐ GET /admin/audit-logs — List audit logs (paginated, filterable)
 - Every state change in the system is logged: entity, action, oldValue, newValue, userId, timestamp, userAgent, IP
 - Filterable: by entity type, action, user, date range, IP
 - Retention: 2 years (regulatory compliance)
 - Sensitive data: password changes logged as event (no values), payment data redacted
- ☐ GET /admin/dashboard — Admin analytics dashboard (booking stats, revenue, user growth)
 - Real-time aggregations:
 - Revenue: total, by period (day/week/month), by provider, by category, by currency
 - Bookings: total, conversion rate (started → completed), cancellation rate, average booking value
 - Users: new registrations, active users (7d/30d), provider signups
 - Provider dashboard (#15): filtered to current provider's data only

- Charts: time series data with configurable granularity (hourly/daily/weekly/monthly)
- ☐ GET /admin/dashboard/bookings — Booking analytics (conversion, cancellation rates)
- ☐ GET /admin/dashboard/revenue — Revenue analytics (by period, provider, category)
- ☐ GET /popular/places — Get most popular places
- ☐ GET /popular/tours — Get most popular tours (by score)
 - Read from analytics.PopularityScores (recalculated every 6 hours)
 - **Popularity Score Formula:**
 - Filterable by: category, place, date range, price range
 - Return: top 50, with score breakdown for admin dashboard
- ☐ GET /provider/analytics — Provider's analytics dashboard
- ☐ GET /trending — Get trending tours (recent popularity spike)
 - Trending = tours with highest score DELTA in last 7 days vs previous 7 days
 - Captures viral / seasonal spikes (e.g., new tour launch, holiday season)
 - Separate from popular (which is absolute, not relative)
- ☐ GET /favorites — List user's favorites (tours + places)
- ☐ GET /favorites/check/{entityType}/{entityId} — Check if entity is in user's favorites
- ☐ POST /favorites — Add to favorites (entityType + entityId)

- EntityType: Tour, Place
- Toggle: if already favorited, return 409 Conflict
- Publish FavoriteAddedIntegrationEvent → used by:
- Discount notifications: when a favorited tour gets a discount, notify user
- Recommendation engine: favorites indicate strong preference signal
- Max favorites: 500 per user (prevent abuse)

☐ DELETE /favorites/{entityType}/{entityId} — Remove from favorites

☐ GET /moderation/logs — Content moderation audit log

☐ GET /reports/admin — List all reports (filterable by status, type)

☐ GET /reviews/admin/flagged — List flagged reviews for moderation

☐ GET /reviews/my-reviews — List current user's reviews

● Tier 2 (Medium)

☐ POST /weather/refresh/{placeId} — Force weather refresh for a place

☐ GET /sitemap.xml — Get generated XML sitemap

- Generated by SitemapRegenerationService every 6 hours
- Includes: all published tours, places, blogs with lastModified dates
- Hreflang alternates for each language (ar, en)
- Sitemap index if >50,000 URLs (split into sub-sitemaps)
- Served from cache (regenerated in background, not on-request)

☐ POST /sitemap/regenerate — Force sitemap regeneration (triggers background job)

☐ GET /map/viewport — Places within map viewport (bounding box)

- Requires: northLat, southLat, eastLng, westLng (bounding box)
- Mapbox integration: return places with coordinates for map pin rendering
- Cluster pins when zoom level is low (>50 results in viewport)
- Include: id, name, lat, lng, primaryImage, rating, tourCount (minimal payload for map)

☐ GET /nearby — Find places near coordinates (lat, lng, radius)

- Requires: lat, lng, radiusKm (max 100km)
- Uses Haversine formula for distance calculation (SQL Server geography or computed)
- Returns places sorted by distance, with distance field in response
- Limit: max 50 results, applies active + soft-delete filters

☐ POST /interactions — Track user interaction (view, click, search, share)

- Fire-and-forget: async processing, non-blocking response
- Interaction types: View, Click, Search, Share, AddToFavorite, BookingStarted, BookingCompleted
- Payload: entityType, entityId, interactionType, metadata (searchQuery, referrer, duration)
- Used by: recommendation engine (collaborative filtering), popularity scoring, analytics dashboards

- Storage: analytics.UserInteractions table (BIGINT auto-increment PK for high volume)
- Rate limit: deduplicate same user + same entity + same type within 5 minutes (prevent view inflation)

☐ DELETE /devices/token/{id} — Unregister device token

☐ POST /devices/token — Register device push token (FCM/APNs)

- Register FCM (Android) or APNs (iOS) token for push notifications
- Deduplicate: update if deviceId already has a token
- Cleanup: remove stale tokens (not updated in 30 days)

☐ DELETE /notifications/{id} — Delete a notification

☐ GET /notifications — List user's notifications (paginated, filterable by type, read/unread)

- Return user's notifications filtered by their preferences
- Types: BookingConfirmed, BookingCancelled, PaymentReceived, NewReview, ReviewReply, ProviderApproved, DiscountOnWishlist, PointsEarned, PointsExpiring, DocumentExpiring, SubscriptionRenewed, TourApproved, DisputeUpdate
- Filter: by type[], read/unread, date range
- Sort: newest first (default), includes relative time ("2 hours ago")
- Pagination: cursor-based (not offset) for real-time consistency

☐ GET /notifications/preferences — Get notification preferences

☐ GET /notifications/unread-count — Get unread notification count

- ☐ POST /notifications/read-all — Mark all notifications as read
- ☐ POST /notifications/{id}/read — Mark notification as read
- ☐ PUT /notifications/preferences — Update notification preferences (per type: in-app, email, push)
 - Per notification type: enable/disable for each channel (inApp, email, push)
 - Default: all enabled for inApp, major events for email, none for push
 - Validation: cannot disable all channels for critical notifications (payment, booking)
- ☐ POST /support/admin/tickets/{id}/assign — Assign ticket to agent
- ☐ POST /support/admin/tickets/{id}/resolve — Mark ticket as resolved
- ☐ GET /support/tickets — List user's support tickets
- ☐ GET /support/tickets/{id} — Get ticket details with messages
- ☐ POST /support/tickets — Create support ticket
 - Categories: Booking Issue, Payment Problem, Provider Complaint, Account Help, Bug Report, Other
 - Priority auto-assignment: Payment = High, Booking = Medium, Other = Low
 - Required: category, subject (10-200 chars), message (20-5000 chars)
 - Auto-assign to available support agent (round-robin)
 - SLA: High = 4h response, Medium = 12h, Low = 24h
- ☐ POST /support/tickets/{id}/close — Close ticket
- ☐ POST /support/tickets/{id}/messages — Add message to ticket

☐ `POST /reports` — Report content (review, blog comment, tour) with reason

- Report reasons: Spam, Inappropriate, Misleading, Harassment, FakeReview, Other
- Required: `entityType`, `entityId`, `reason`, `description` (20-500 chars)
- Rate limit: max 10 reports per user per day (prevent report abuse)
- Auto-action: if 3+ reports on same content → auto-hide, escalate to moderation
- Moderation log: every action creates `ContentModerationLog` entry for audit trail

☐ `POST /reports/admin/{id}/resolve` — Resolve report (dismiss, warn, remove content)

☐ `POST /reviews` — Create review (rating 1-5, text, optional photos) — requires verified booking

- **Verified booking requirement:** user must have a `Completed` booking for this tour/place
- Rating: 1-5 integer, required
- Text: 20-2000 characters, optional but encouraged (10 bonus loyalty points for text review)
- Photos: max 5, must be JPEG/PNG/WebP, max 5MB each
- **Profanity filter:** scan text against blocklist + ML-based detection, auto-flag if suspicious
- One review per user per booking (prevent spam)
- Review appears immediately but flagged ones go to moderation queue

- Publish ReviewCreatedIntegrationEvent → recalculate tour rating, notify provider

☐ DELETE /reviews/{id} — Delete own review

☐ PUT /reviews/{id} — Edit own review (within 48h window)

- Allowed only within 48 hours of creation
- Re-run profanity filter on updated text
- Track edit history (keep original for moderation)

☐ POST /reviews/{id}/reply — Provider replies to review

- One reply per review (can edit, not add multiple)
- Provider must own the tour being reviewed
- Profanity filter on reply text
- Notify reviewer that provider replied

☐ POST /reviews/{id}/report — Report inappropriate review

☐ POST /reviews/admin/{id}/approve — Approve flagged review

☐ POST /reviews/admin/{id}/remove — Remove review (moderation)

Tier 3 (Hard)

☐ GET /search — Full-text search tours (with filters, sorting, pagination)

- Full-text search across: title, description (all languages), place name, category name, tag names
- Ranking factors: text relevance, popularity score, review count, recency

- Faceted search: return counts per category, price range distribution, available dates
- Search suggestions: autocomplete endpoint (debounced, returns top 5)

Infrastructure & Background Jobs

☐ **Background Service:** SlotLockCleanupService (Booking)

- Delete expired SlotLocks (TTL = 10 min). Releases unpaid reservation slots back to availability.

☐ **Background Service:** BookingAutoExpireService (Booking)

- Auto-cancel bookings stuck in AwaitingPayment for >10 min.

☐ **Background Service:** ProviderAutoAcceptService (Booking)

- Auto-confirm bookings waiting provider confirmation for >24h (non-instant booking).

☐ **Background Service:** DocumentExpiryCheckService (Accounts)

- Check ProviderDocuments.ExpiresAt against today. Mark expired docs, notify provider.

☐ **Background Service:** PayoutBatchingService (Finance)

- Weekly Sunday midnight. Aggregate completed bookings into provider payout batches.

☐ **Background Service:** RefundRetryService (Finance)

- Every 15 minutes. Retry failed refund transactions.

☐ **Background Service:** SitemapRegenerationService (SEO)

- Every 6 hours. Regenerate XML sitemap from all published content.

☐ **Background Service:** WeatherPreFetchService (SEO)

- Daily. Pre-fetch weather data for top 50 tour locations.

☐ **SignalR Hub:** NotificationHub (Messaging)

- WebSocket hub for real-time bell icon push notifications.
-